

Wavelet Transform and Spatial implementations

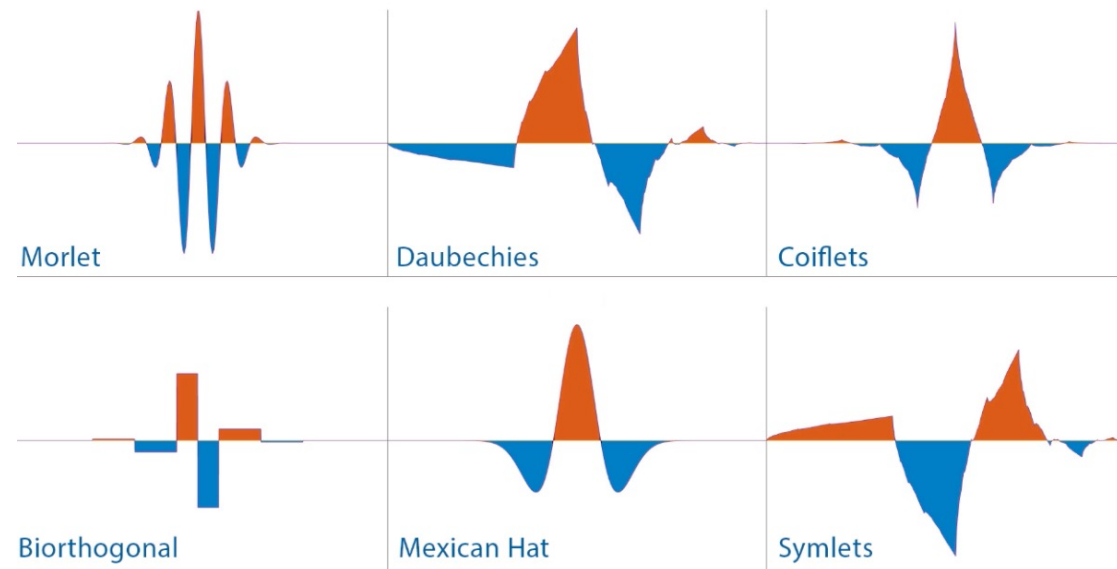
Maria Jose Rueda

Wavelet

Images have smooth regions interrupted by edges (high frequency) or abrupt changes in contrast. These abrupt changes are often the most interesting parts of the data, both perceptually and in terms of the information they provide.

- Wavelet: (little waves) functions that are well localised in time and frequency. They have finite duration.
- Fourier transform: sum of sine waves, which are not localised in time or space. They oscillate forever.

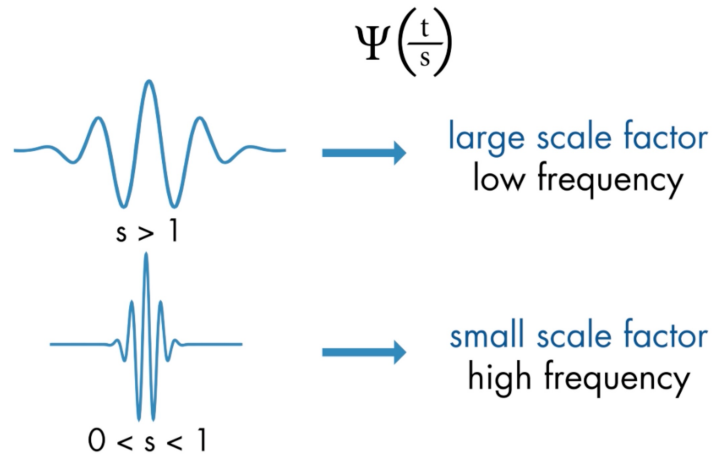
Different wavelets for different wavelet analysis:



We start with one of these wavelets and then this signal is scaled and sifted forming new versions for analysing the input signal.

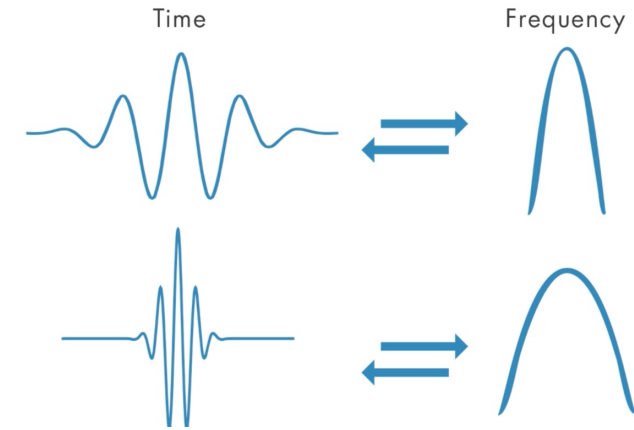
Wavelet concepts

- **Scaling:** how much is stretching or shrinking the signal in time. Inversely proportional to frequency.

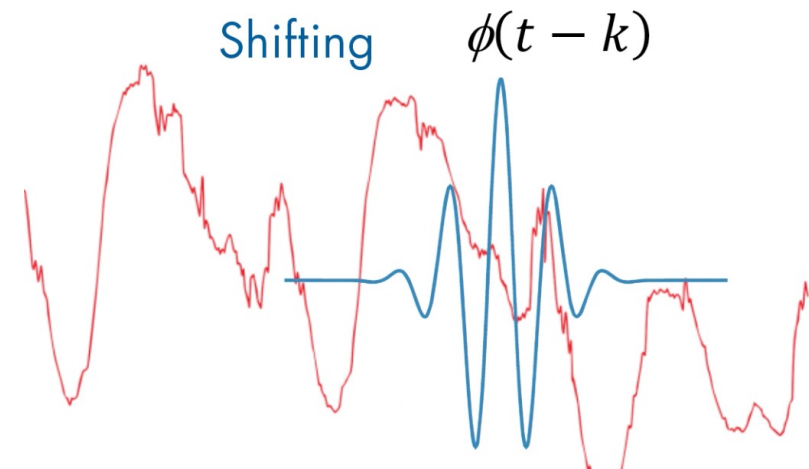


- **Shifting:** delay or advance the wavelet along the length of the signal. (Location in space or time)

Our signal can be decomposed using the same wavelet with different scales and shift.



Stretched wavelet helps in capturing the slowly varying changes and a compressed wavelet helps in capturing abrupt changes.



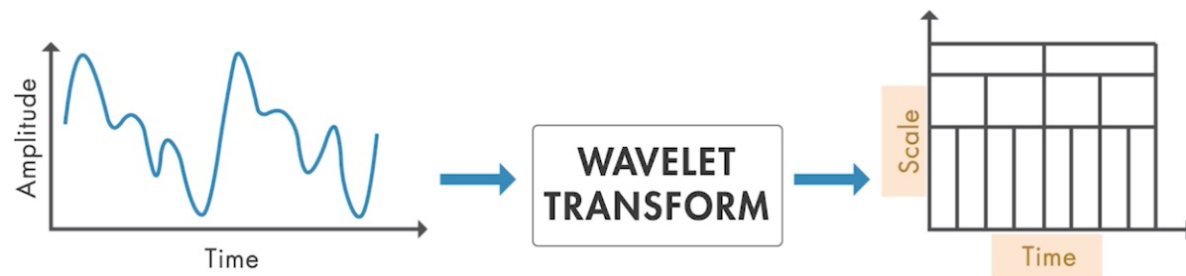
Wavelet transform types

*Differ in how they discretize the **scale** and the **translation** parameters.*

CONTINUOUS WAVELET TRANSFORM

This transform is used to obtain a **simultaneous** time frequency analysis of a signal. The output of CWT are coefficients (a function of scale or frequency and time).

Application: time frequency analysis, and filtering of time localized frequency components.



WAVELET SCALE	2	4	8	16
EQUIVALENT FREQ (F_{eq})	$\frac{F_{eq}}{2}$	$\frac{F_{eq}}{4}$	$\frac{F_{eq}}{8}$	$\frac{F_{eq}}{16}$

Better characterize oscillatory behaviour in signals with the Continuous wavelet transform: 1000 samples analysed with 20 scales results in 20,000 coefficients

Allow fine scale analysis

Wavelet transform types

*Differ in how they discretize the **scale** and the **translation** parameters.*

DISCRETE WAVELET TRANSFORM

The **base scale** in DWT is set to **2**. You can obtain different scales by: 2^j ($j = 1, 2, 3, 4 \dots$)

Dynamic scaling and shifting: 2^{jm} ($m = 1, 2, 3, 4 \dots$)

Scaling

Translation

(This sampling eliminates redundancy in coefficients good for compressing signals)

The length of the coefficients in each sub-band is half of the number of coefficients in the preceding stage:

Input Image
1024 x 1024

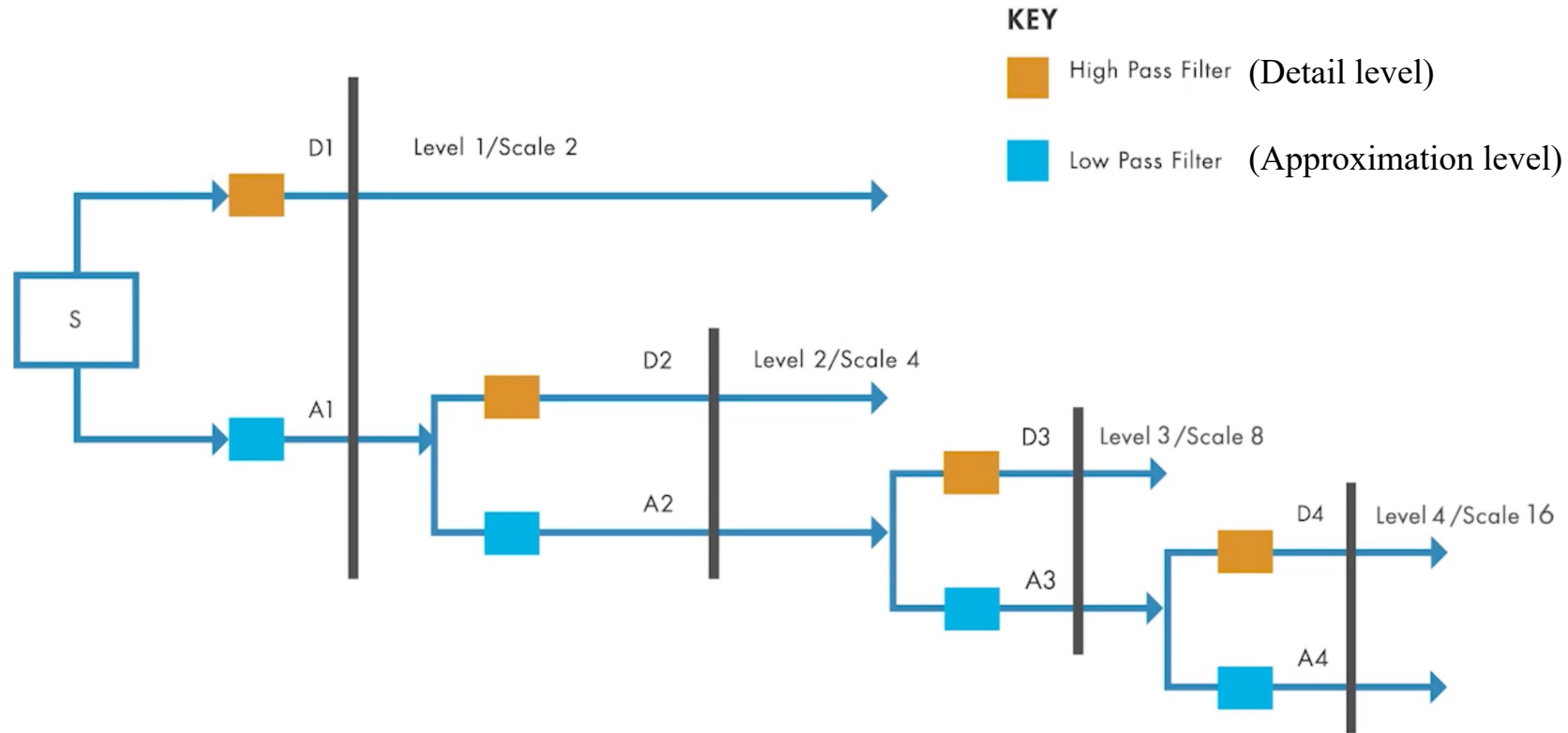
Lvl 1:	512 x 512 x 3
Lvl 2:	256 x 256 x 3
Lvl 3:	128 x 128 x 3
Lvl 4:	64 x 64 x 3
Lvl 5:	32 x 32 x 3
Lvl 6:	16 x 16 x 3
Lvl 8:	4 x 4 x 3
Lvl 9:	2 x 2 x 3
Lvl 10:	1 x 1 x 3

Coefficients

DWT process

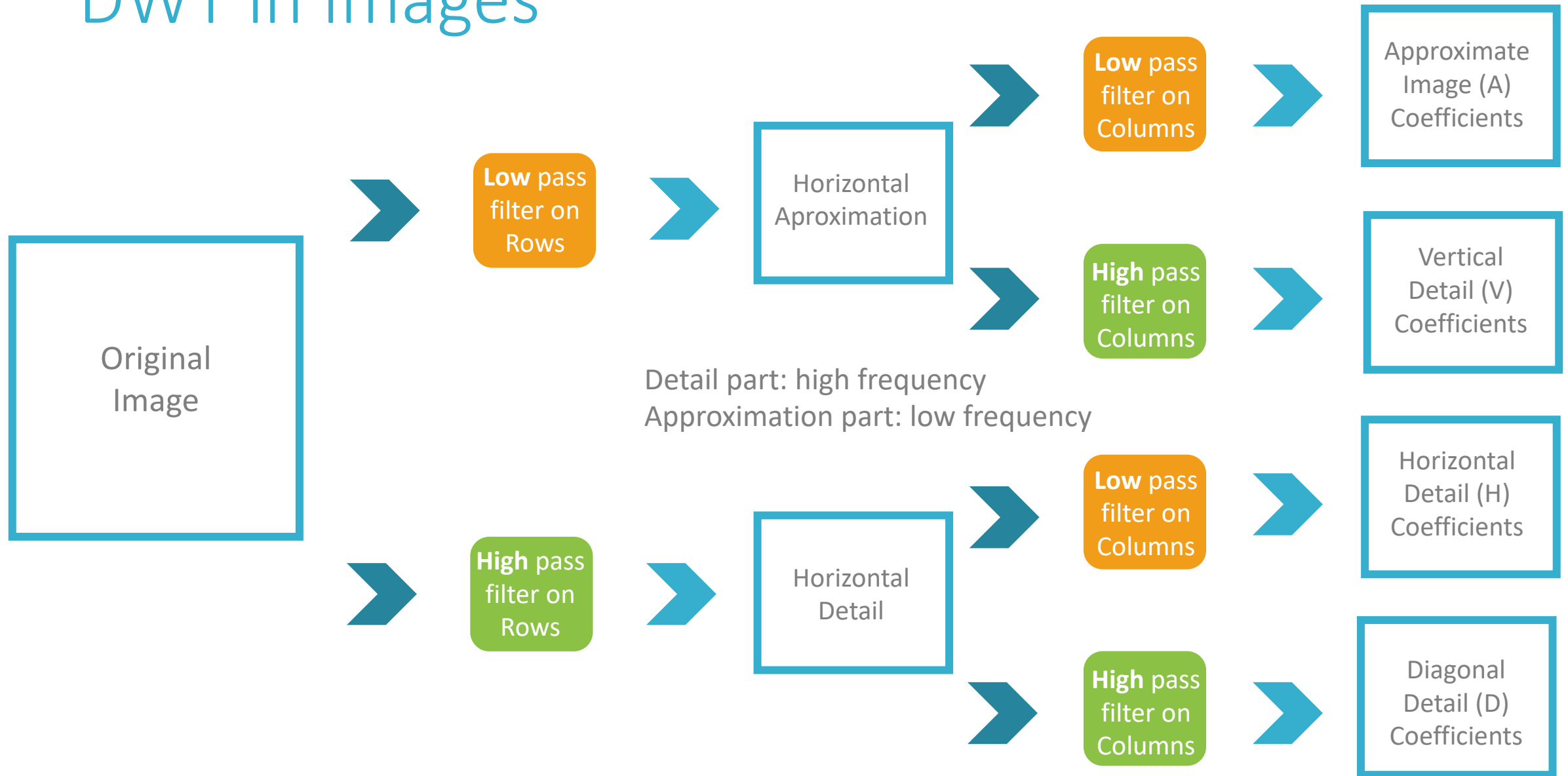
This process is equivalent to comparing the signal with discrete filters bands.

Signal S is filtered with high-pass and low-pass filter → Obtain low-pass and high-pass sub-bands



DWT helps analyse signals at progressively narrower sub-bands at different resolutions. It also helps denoise and compress signals.

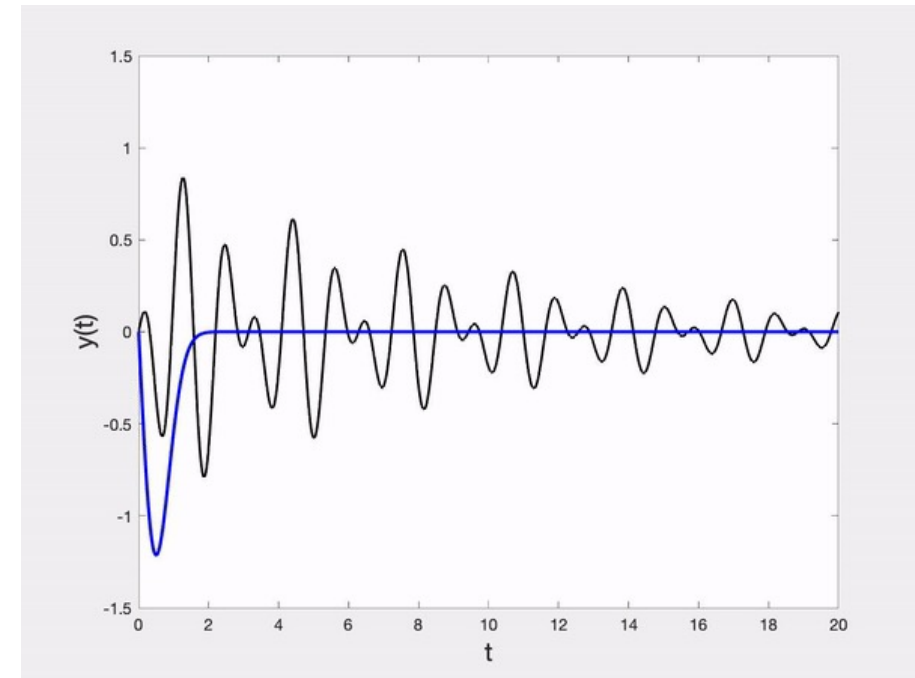
DWT in Images



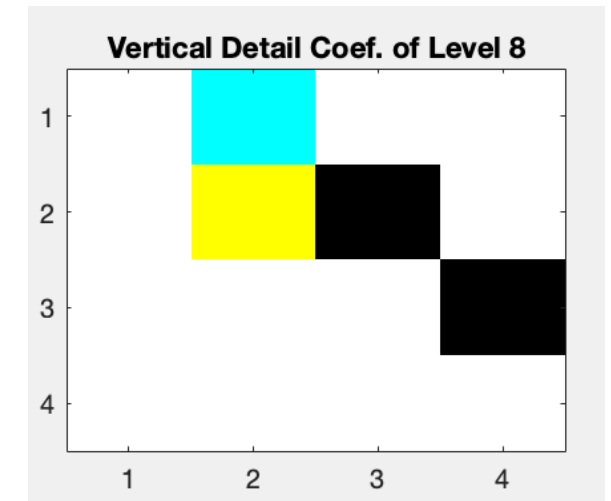
DWT coefficients

DWT compute how much of the wavelet (with specific scale and location) there is in the input signal. In other words, a set of different scales wavelets are convolved through the signal.

At each time step the signal and the wavelet are multiplied, obtaining the correspondent coefficient at that time and that scale.

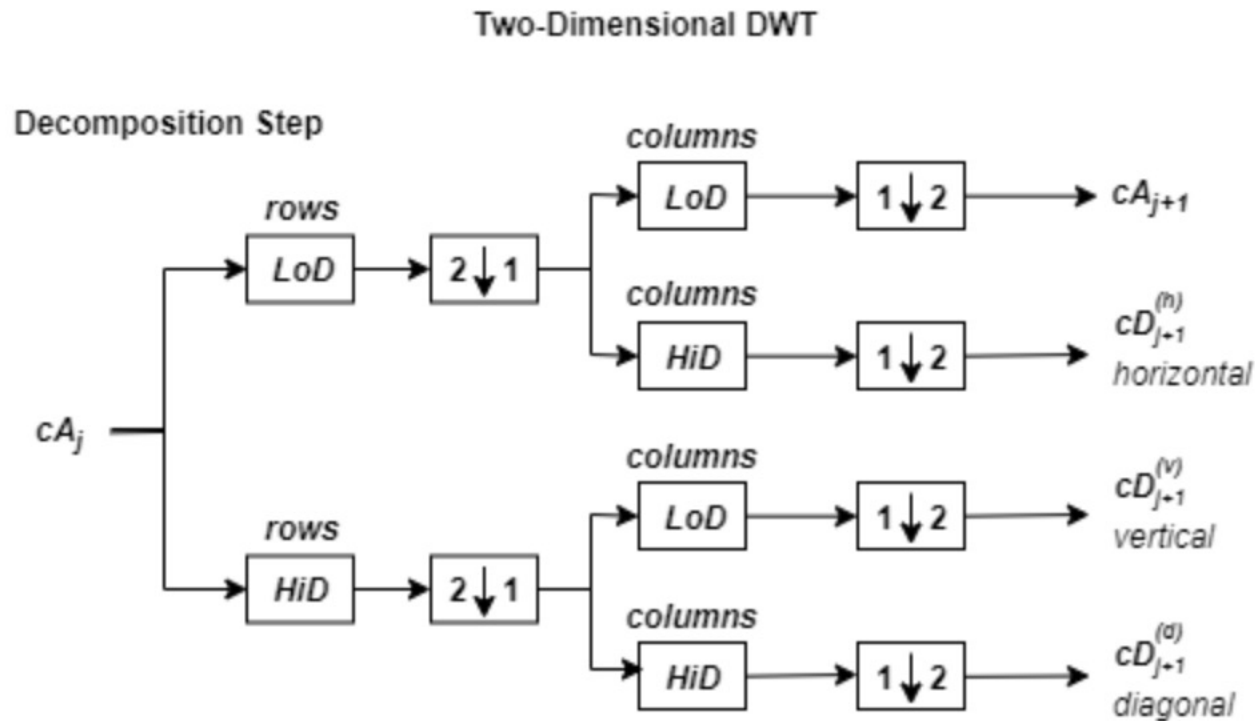


```
val(:,:,1) =  
1.0e+04 *  
    0.1832   -0.0322    0.0783    1.0249  
    0.5245    0.1091   -0.2373    0.4848  
    0.2761    0.6634    0.3700   -0.4715  
    0.6819    0.2851    0.0852    0.0421  
val(:,:,2) =  
1.0e+04 *  
    0.0983    0.0952    0.1149    1.1201  
    0.5700    0.0374   -0.2423    0.3644  
    0.3707    0.6577    0.3873   -0.4893  
    0.7615    0.2701    0.0324    0.0284  
val(:,:,3) =  
1.0e+04 *  
    0.0469    0.1884    0.1348    1.1906  
    0.7063   -0.0141   -0.2658    0.3356  
    0.5179    0.6357    0.3489   -0.5124  
    0.8285    0.2561    0.0005    0.0138
```



DWT MatLab notation

The chart describes the basic decomposition step for images:



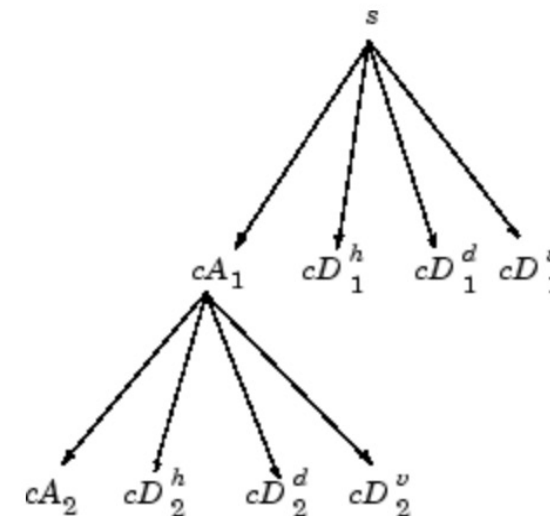
where

- $\begin{bmatrix} 2 \downarrow 1 \end{bmatrix}$ – Downsample columns: keep the even-indexed columns.
- $\begin{bmatrix} 1 \downarrow 2 \end{bmatrix}$ – Downsample rows: keep the even-indexed rows.
- $\begin{matrix} \text{rows} \\ \boxed{X} \end{matrix}$ – Convolve with filter X the rows of the entry.
- $\begin{matrix} \text{columns} \\ \boxed{X} \end{matrix}$ – Convolve with filter X the columns of the entry.

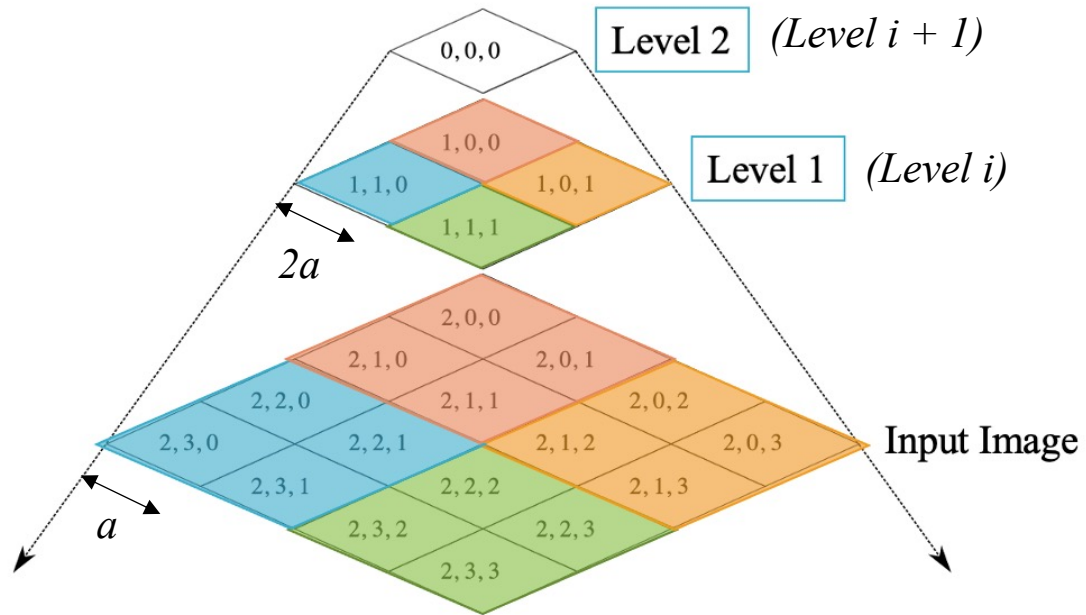
and

Initialization: $cA_0 = s$.

So, for $J = 2$, the two-dimensional wavelet tree has the form



DWT Levels

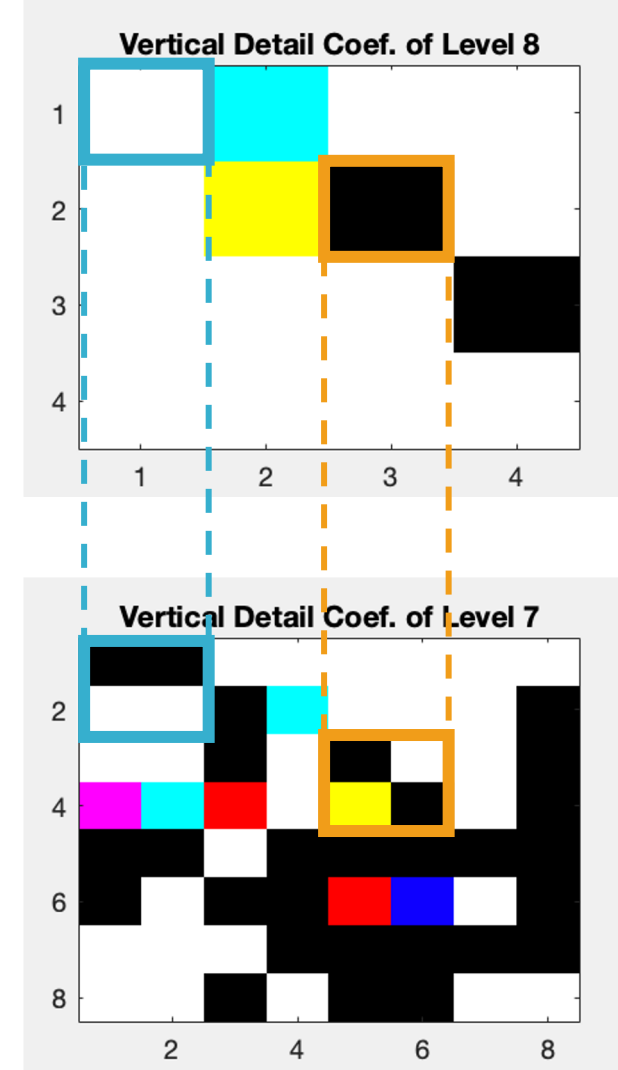


$$\text{Spatial frequency} = \frac{1}{2^i a}$$

$$\text{Scale} = 2^i$$

If we know the scale, we can obtain the frequency.

With DWT we can get the contrast.



DWT Implementation

Next steps:



Obtain the maximum value between V, H, D, A coefficients for each pixel in each level



Save the level of the corresponding maximum value in each pixel location



Obtain the frequency



Calculate the frequency (cycle/degree) which will serve as input for Barten's model

For obtaining the pixel location with the level i :

Pixel location in the input image: (x, y)

Level: i

$$\text{Pixel location: } (\lceil \frac{x}{2^i} \rceil, \lceil \frac{y}{2^i} \rceil)$$

Old frequency implementation:

```
% fq_lvl = a/(2^i);  
% fq = fq_lvl/(2*atan(0.5/d));
```

New frequency implementation:

```
% Obtain spatial frequency with the level  
% a: pixel size  
% d: distance from the screen  
% fq_lvl: Hz  
% fq_spat: cycles/pixel  
% fq: cycles/degree  
scale = 2^i;  
fq_lvl = scal2frq(scale, 'haar');  
fq_spat = fq_lvl/(2^i);  
degree = 2*atan((a/2)/d);  
fq = fq_spat/degree;
```

The size of the filter per level is 2^i (i: level), so we can obtain the frequency per pixel (cycles/pixel):

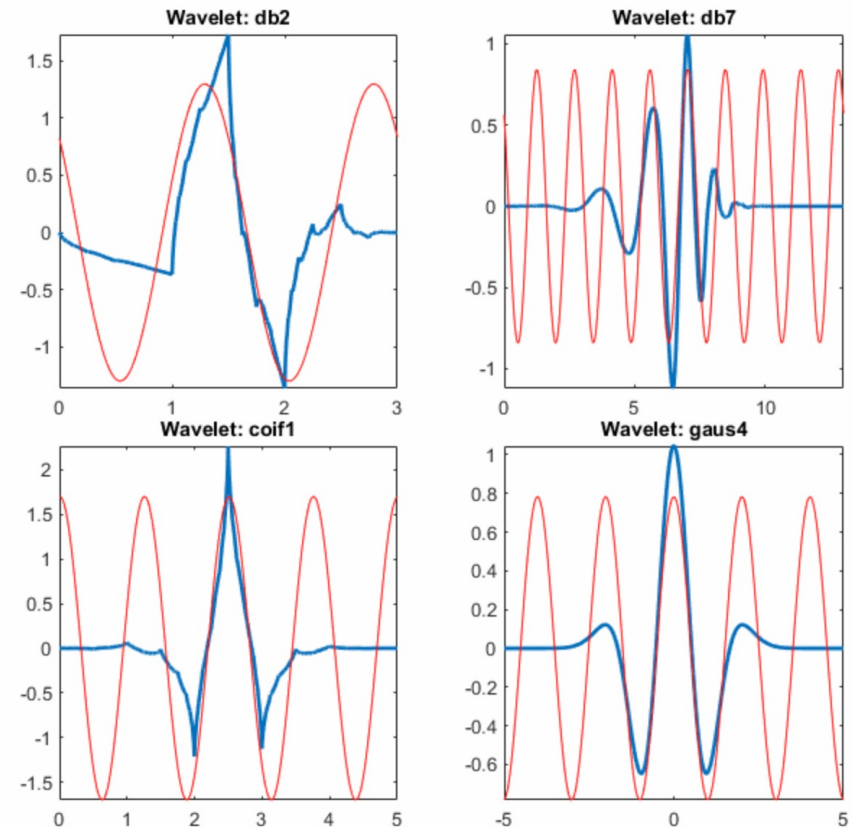
$\text{fq_spat} = \text{fq_lvl}/2^i;$

②

With this function you can obtain the frequency which depends on the selected wavelet, in this case 'haar'. The output is Hz (cycles/sec)

①

```
scale = 2^i;  
fq_lvl = scal2frq(scale, 'haar');
```

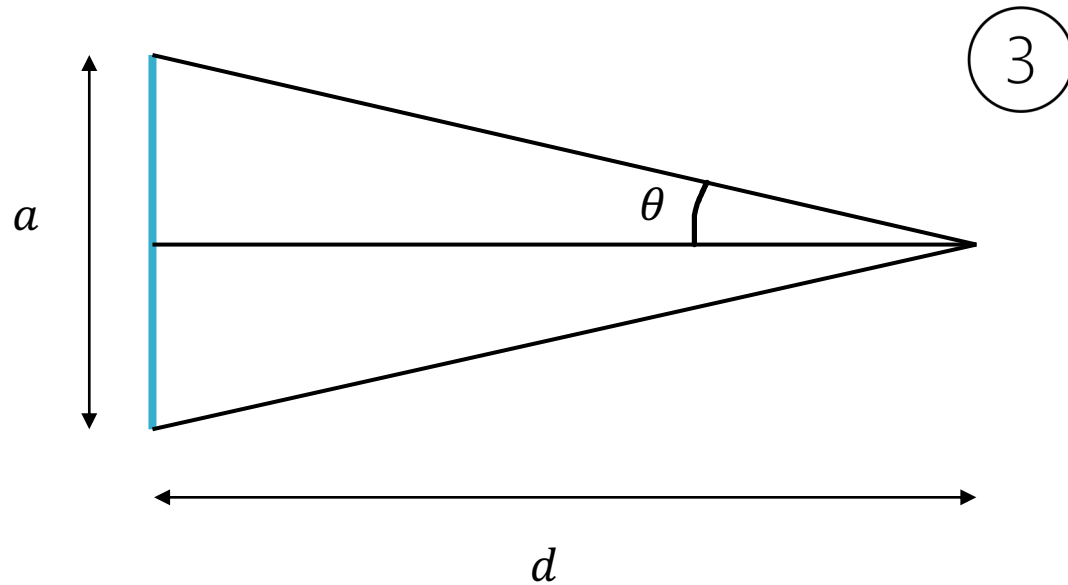


From cycles/pixel to cycles/degree:

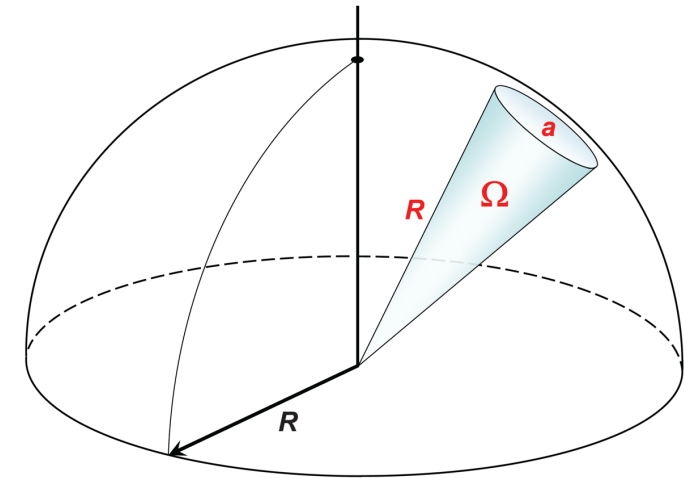
$$\frac{\text{cycles}}{\text{pixel}} * \frac{1 \text{ pixel}}{\text{degrees}} = \frac{\text{cycles}}{\text{degree}}$$

$$\text{degree} = 2 * \text{atan}((a/2)/d);$$

$$\text{fq} = \text{fq_spat}/\text{degree};$$



$(1 / (2^i * a))$ because the angle that the pixel forms with the line of sight (to calculate the frequency) changes depending on the level, since the window size covers more pixels in each level, the pixel size will be twice smaller



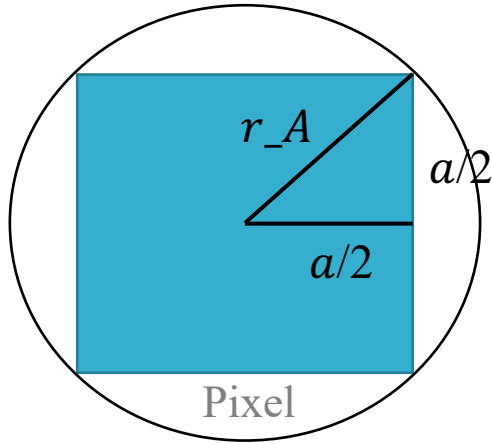
Angular object area in square degrees implementation:

```
% r_A: radius in m (pixel surface)
% r_d: radius sphere in m (calculated with the distance d)
% sr: solid angle in square steradians
% X_0: angular object area in square degrees
r_A = sqrt((a/2)^2+(a/2)^2);
A = pi*(r_A^2);
r_d = sqrt((a/2)^2+(d^2));
sr = A/(r_d^2);
X_0 = rad2deg(sr);
```

In this case a is the spherical surface area and R is the radius of the sphere considered. The solid angle is:

$$\Omega = \frac{A}{R^2}$$

Spherical surface area of the pixel:



$$r_A = \sqrt{(a/2)^2 + (a/2)^2}; \quad (1)$$

$$A = \pi r_A^2$$

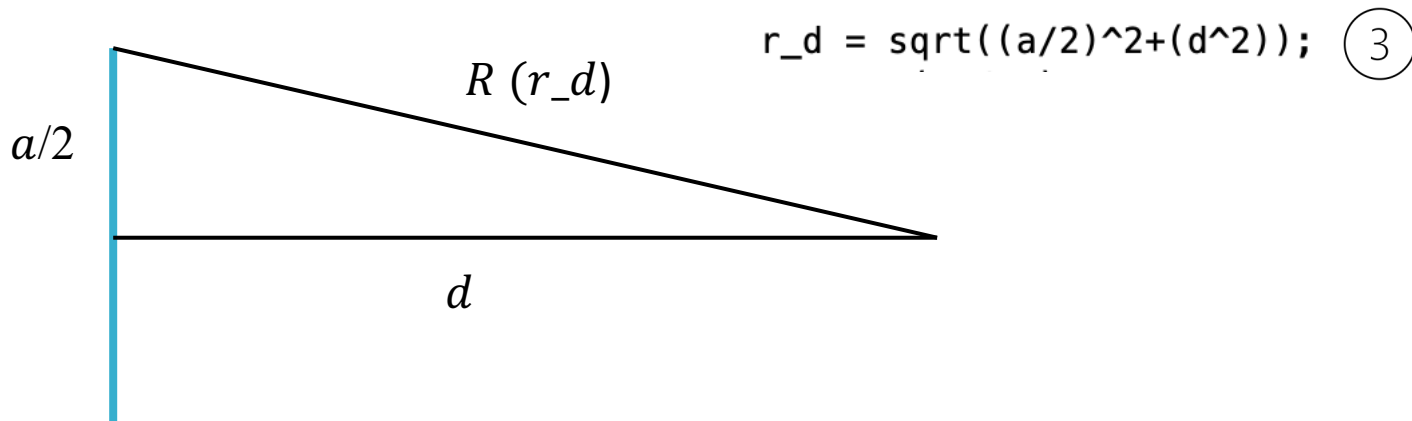
$$A = \pi * (r_A^2); \quad (2)$$

Solid angle in steradians:

$$\Omega = \frac{A}{R^2}$$

$$sr = A / (r_d^2); \quad (4)$$

Radius of the sphere considered :



$$r_d = \sqrt{(a/2)^2 + (d^2)}; \quad (3)$$

From sr to degrees:

$$X_0 = \text{rad2deg}(sr);$$